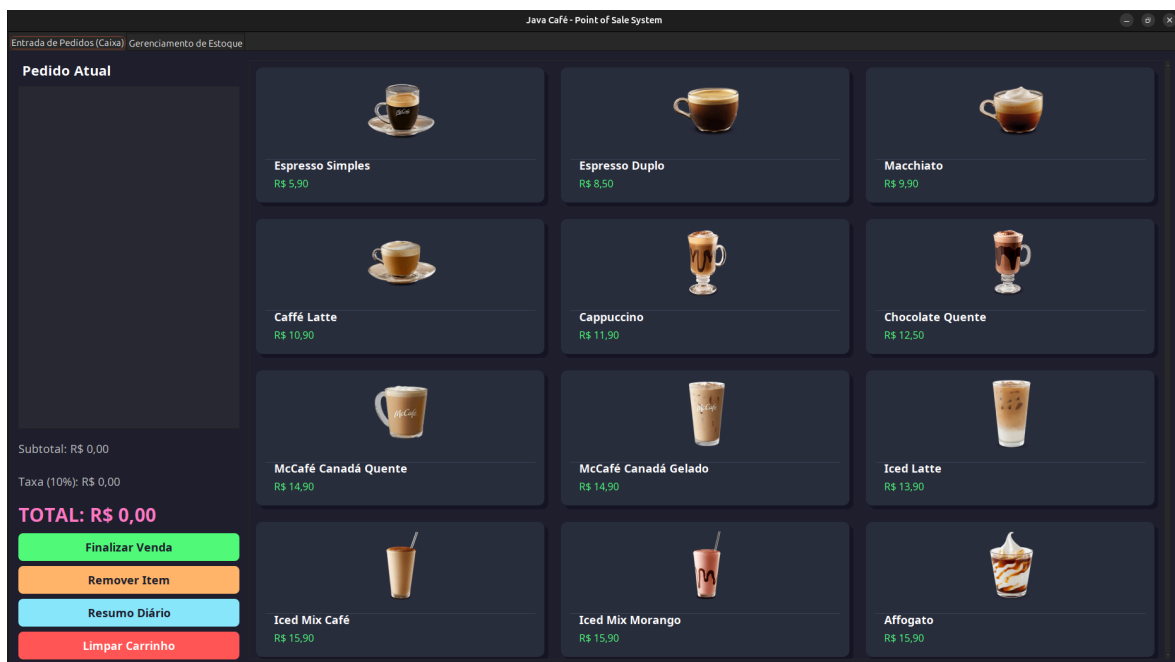




UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E COMPUTAÇÃO
DEPARTAMENTO DE SISTEMAS DE COMPUTAÇÃO

PROJECT REPORT - JAVA CAFÉ

Disciplina: SCC0504 - Programação Orientada a Objetos



GROUP IDENTIFICATION:

Giovana Rafaela Marmo de Almeida

Júlio César Tanaka Vergamini (Líder)

Laysa Almeida Oliveira

Nº USP: 15652896

Nº USP: 15466276

Nº USP: 14588002

SUMMARY

1. REQUIREMENTS.....	3
2. PROJECT DESCRIPTION.....	3
3. COMMENTS ABOUT THE CODE.....	5
4. TEST PLAN.....	6
5. TEST RESULTS.....	6
6. BUILD PROCEDURES.....	7
7. PROBLEMS.....	8
8. COMMENTS.....	8

LINK GITHUB: <https://github.com/Tanaka-txt/Java-Cafe>

1. REQUIREMENTS

O sistema foi desenvolvido seguindo estritamente as especificações da Versão Compacta (SCC0504) do projeto "Java Café". Todos os 6 requisitos obrigatórios foram atendidos:

1. **User Interface:** Criação de uma GUI com duas visões principais (*Order Entry* e *Inventory*). O painel de vendas exibe os itens como botões dinâmicos e mantém o valor total da compra visível a todo momento.
 2. **Order Processing:** Implementação da lógica para adicionar e remover itens individuais do pedido, com cálculo automático de subtotal, taxa (10%) e total. A finalização da venda deduz o estoque em tempo real e exibe um recibo em texto numa caixa de diálogo (*JOptionPane*).
 3. **Inventory Management:** O sistema mantém uma lista de produtos (nome, preço e quantidade), permite a atualização manual do estoque pela equipe e deduz quantidades automaticamente após cada venda.
 4. **Daily Sales Summary:** Inclusão de recursos para visualizar a receita total arrecadada no dia e a contagem de transações.
 5. **Data Persistence:** Os registros de inventário e vendas são salvos em arquivos locais (estoque.csv e vendas.csv) e recarregados corretamente na inicialização da aplicação, mantendo o estado entre as sessões.
 6. **Exception Handling:** Criação da exceção customizada *OutOfStockException*, que captura e exibe mensagens de erro amigáveis ao usuário caso haja tentativa de venda de itens sem estoque, substituindo as *stack traces* brutas no terminal.
- **Optional Enhancements (Bônus implementados):** Foi implementado um sistema de alerta para estoques baixos (itens com 11 unidades ou menos), exibido no painel de inventário. Além disso, a aplicação conta com melhorias visuais não exigidas, como um layout em "Dark Mode", botões com bordas arredondadas e uma lógica de redimensionamento proporcional automático para as imagens dos produtos.

2. PROJECT DESCRIPTION

Para organizar o projeto, nós separamos o código em diferentes pacotes (pastas). A ideia principal foi garantir que a parte visual (o código que desenha as telas e botões) não ficasse

misturada com a parte lógica (o código que faz os cálculos de taxas e a verificação do estoque). O código foi dividido da seguinte forma:

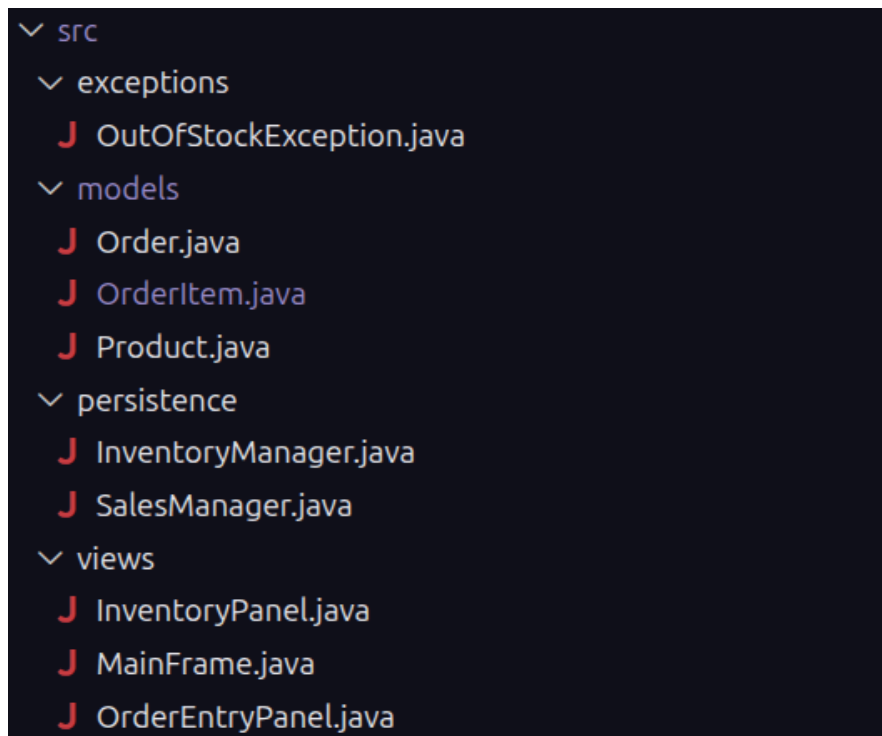


Figura 1: Estrutura de pacotes do projeto no ambiente de desenvolvimento, ilustrando a separação do código.

- **Pacote models:** É onde ficam os moldes dos elementos que existem no café (as classes *Product*, *Order* e *OrderItem*). É este pacote que guarda as informações do que está sendo comprado e realiza as contas matemáticas de soma e porcentagem.
- **Pacote exceptions:** É a pasta destinada ao aviso de erro personalizado (*OutOfStockException*), criada para organizar e tratar o que acontece quando o estoque de um produto acaba.
- **Pacote persistence:** Aqui ficam os arquivos (*InventoryManager* e *SalesManager*) que têm a função de salvar os dados e ler os arquivos .csv. Eles garantem que os registros dos produtos e das vendas sejam mantidos de forma segura quando o programa for fechado.
- **Pacote views:** Gerencia a interface com o usuário. É onde está organizado todo o código visual. A classe principal *MainFrame* contém as abas do sistema, e os painéis *OrderEntryPanel* e *InventoryPanel* constroem as tabelas, os botões coloridos e as imagens para a interação do usuário.



Figura 2: Interface gráfica da aba de Gerenciamento de Estoque, exibindo a listagem de produtos, os botões de interação e o painel com o resumo financeiro diário.

3. COMMENTS ABOUT THE CODE

O tratamento de erros foi estruturado em etapas para não misturar os cálculos com a parte visual. Primeiro, a classe *OutOfStockException* foi criada apenas para guardar a mensagem personalizada do problema.

```

J OutOfStockException.java X
JavaCafe > src > exceptions > J OutOfStockException.java > ...
Tanaka-txt, last week | 1 author (Tanaka-txt)
1 package exceptions;
2
Tanaka-txt, last week | 1 author (Tanaka-txt)
3 public class OutOfStockException extends Exception {
4     public OutOfStockException(String message) {
5         // Repassa a mensagem de erro para a classe pai (Exception)
6         super(message);
7     }
8 }

```

O disparo desse erro (throw) acontece direto na classe *Product*, que faz a verificação do estoque e lança a exceção caso a quantidade pedida seja maior que a disponível. Por fim, a tela *OrderEntryPanel* faz o tratamento (catch) capturando esse erro. Assim, em vez de o programa fechar de forma inesperada ou mostrar códigos no terminal, ele abre uma janela de aviso amigável (*JOptionPane*) para o usuário.

Além disso, foi implementada uma verificação dinâmica de caminhos de diretório para o carregamento das imagens dos produtos (if (!imgFile.exists())).

```
File imgFile = new File("imagens/" + product.getName() + ext);  
  
if (!imgFile.exists()) {  
    imgFile = new File("JavaCafe/imagens/" + product.getName() + ext);  
}
```

Isso foi feito para garantir a compatibilidade de execução do projeto em diferentes IDEs (como Eclipse e VS Code), que frequentemente adotam pastas raízes distintas durante a compilação do projeto, evitando que caminhos relativos quebrem a interface.

4. TEST PLAN

O plano de testes consistiu em simular o uso real na interface gráfica para ver se as contas e os bloqueios estavam funcionando:

- Adição e remoção de múltiplos itens no carrinho para verificar a exatidão dos cálculos contínuos de subtotal, taxas e total.
- Tentativa de finalização de vendas solicitando uma quantidade superior à disponível no estoque, visando acionar e validar o bloqueio do sistema.
- Cadastro de novos produtos, adição de estoque e limpeza de vendas do dia, seguidos de reinicializações completas da aplicação, para confirmar a correta persistência dos dados nos arquivos locais.

5. TEST RESULTS

Os resultados dos testes demonstraram que a lógica matemática do carrinho é atualizada em tempo real de forma precisa. A exceção de controle de estoque barrou com sucesso as transações inválidas, exibindo as mensagens corretas sem interromper o fluxo do software. A persistência de dados mostrou-se estável: os arquivos .csv registraram as atualizações perfeitamente e o sistema demonstrou capacidade de recriar o estoque padrão automaticamente caso o arquivo principal de inventário seja deletado.

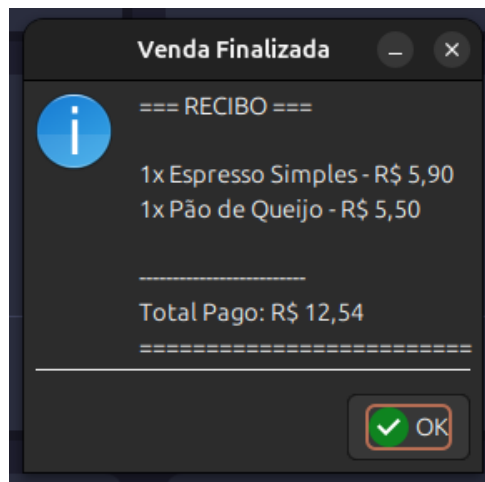


Figura 3: Demonstração da geração de recibo interativo após a conclusão de uma venda



Figura 4: Painel financeiro atualizado em tempo real e exibição do alerta visual para produtos com o estoque abaixo do limite seguro.



Figura 5: Tratamento de erro interativo exibido ao usuário, bloqueando a tentativa de venda de um produto com quantidade superior ao estoque disponível.

6. BUILD PROCEDURES

Para compilar e executar o projeto, devem ser seguidos os seguintes passos:

1. Garantir que o ambiente de desenvolvimento Java (JDK) esteja instalado e devidamente configurado na máquina.
2. Clonar o repositório do projeto via Git utilizando o terminal:
`git clone [COLOQUE_AQUI_O_LINK_DO_GITHUB]`

(ou efetuar o download do código-fonte em formato ZIP).

3. Abrir a pasta raiz do projeto clonado (JavaCafe) em uma IDE compatível de sua preferência (Eclipse, VS Code ou IntelliJ).
4. Navegar até o pacote views e executar a classe MainFrame.java. A interface gráfica será iniciada automaticamente. *(Nota: Não é necessária a configuração prévia de bancos de dados ou servidores externos, pois o sistema cria e consome seus próprios arquivos de texto).*

7. PROBLEMS

O principal desafio foi a diferença de configuração entre IDEs (VS Code e Eclipse), que gerou conflitos na leitura dos caminhos relativos das imagens. O problema foi resolvido implementando uma verificação dupla de diretórios diretamente no código, garantindo que as imagens carreguem perfeitamente em qualquer ambiente.

8. COMMENTS

O projeto permitiu aplicar os conceitos de Orientação a Objetos na prática. A criação da interface gráfica (Swing) integrada à manipulação de arquivos evidenciou a importância de separar o código em pacotes estruturados. O resultado foi um sistema de Ponto de Venda organizado, de fácil manutenção e totalmente funcional, cumprindo os objetivos propostos.